

Innovation Lab Management

Final Project Report (Vitalism Core)

Course: Acertar o Rumo
Edition: 12th Edition
Group: Group 3 AOR
Supervisor: João Braz Simões
Date: July 5, 2026

Authors

Francileide Vasconcelos · Luis Ferreira · Tiago Pancas

Contents

1	Introduction & Project Framework	1
1.1	Context & Objectives	1
1.2	Scope & Report Structure	1
2	Requirements Analysis	2
2.1	Functional & Non-Functional Analysis	2
2.2	User Profiles & Permitted Access Matrix	2
2.3	Assumptions & Constraints	3
3	System Architecture & Engineering Design	3
3.1	Architectural Overview & Technology Choice	3
3.2	Architectural Decision Records (ADRs)	3
3.3	Model Blueprints (Domain, DB, & API Engineering)	4
4	Implementation & Quality Infrastructure	4
4.1	Core Components Development	4
4.2	MiniDSL & Rules Evaluation Engine	4
4.3	Requirement Traceability Matrix	5
5	Verification, Deployment, & Operations	5
5.1	Testing Strategy & Static Code Analysis	5
5.2	Operational Infrastructure Setup	5
5.3	Risk Matrix & Mitigations	6
6	Bonus Modules, Project Evaluation, & Conclusions	6
6.1	Bonus Features Implementation	6
6.2	Challenges, Extensions, & Closing Summary	6
A	Technical Blueprints & Database DDL	7
B	System Screenshots & User Manual	9
C	Automated Quality Reports	11

List of Figures

1	Ecosystem Class Diagram (Mirror of <code>domain.entity</code>)	8
2	Entity-Relationship Diagram (Mirror of <code>schema.sql</code>)	8
3	Main Dashboard - Monitoring Interface and Workflow	9
4	2D Representation of the Human Body in Alert/Critical State	10
5	Clinical Evaluation Technical Report - <i>Batch</i> /Static File Scenario	10
6	Clinical Evaluation Technical Report - <i>Live/Stream</i> Network Scenario	11

List of Tables

1	Permissions Matrix by User Profile	2
2	System REST API Endpoint Mapping Rules	4
3	Functional Requirements Traceability Configuration (Excerpt)	5
4	System Risk Register Matrix	6
5	Official Metrics and Development Scope	7

1 Introduction & Project Framework

1.1 Context & Objectives

An in-depth understanding of how the human body functions, both under normal and pathological conditions, is a determining factor for the evolution of medical training, scientific research, and clinical practice. The ability to observe, analyze, and simulate real physiological reactions in a controlled and reproducible manner is essential to support decision-making, learning, and the validation of interventions in complex and high-risk contexts. However, modeling these physiological scenarios is a complex process that requires robust tools capable of integrating the multiple systems of the human body.

It is within this framework that the need arose to develop the **Innovation Lab Management (Vitalism Core)** project as part of the 12th edition of the Acertar o Rumor program. The general objective of this project is the design and implementation of a web-based application that allows viewing and tracking the physiological evolution of a virtual patient, leveraging advanced simulation engines. To this end, the solution was designed to implement the following core features:

- I. Execution of clinical physiological scenarios for a virtual patient;
- II. Continuous monitoring of the patient's condition, processing and presenting data on physiological systems over time;
- III. Visual highlighting, on a 2D representation of the human body, of the impacted physiological systems;
- IV. Alert mechanisms based on physiological events, in order to signal relevant changes in the patient's state, facilitating the understanding of the effects of different scenarios.

Core Domain Definitions

Simulation

Process in which the application receives and displays the evolution of the virtual patient's clinical state over time, in response to the evaluation engine.

Clinical Scenario

Set of parameters and events that define the simulated medical situation (e.g., cardiac arrest or hypovolemic shock).

Physiological Reading

Value associated with a vital sign collected at a given timestamp (e.g., Heart Rate or Oxygen Saturation - SpO2).

1.2 Scope & Report Structure

The operational scope of the platform focuses on the uninterrupted ingestion and processing of vital data. The system was calibrated to simultaneously support the bulk loading of static files (*Batch Payloads* in CSV) for analyzing past simulations, and real-time telemetry processing (*Streaming Payloads*) via WebSockets. To guarantee server stability during peak data reception, the operational limit of ingestion was fixed at a maximum buffer of 50,000 events retained in memory.

Due to the strict page limit imposed for this final project, the extensive technical documentation produced throughout development was heavily condensed. The main narrative is structured into six vital sections (from architectural decisions to code quality and final conclusions), transferring extensive screenshots, complex database scripts (DDL), and analytical coverage reports to the appendices. This strategic condensation allows keeping the main body focused, direct, and fully within the required limit.

2 Requirements Analysis

2.1 Functional & Non-Functional Analysis

The requested core features were deconstructed and rigorously aligned with engineering quality parameters:

- **Authentication and Validation:** Implementation of secure sessions with XSS-protected Cookies. Additionally, all submissions pass through strict **input validation filters** in the backend to prevent malicious injections before reaching the database.
- **Live Updates (Real-Time):** Integration of WebSockets (STOMP) to ensure the medical interface updates instantly to telemetry streams.
- **Rules Engine:** A flexible text-based system capable of triggering alerts upon vital sign variations, evaluating the temporal window of anomalies.
- **Non-Diagnostic Report Format:** The final application generates a final audit report (in PDF) focused purely on simulation metrics, legally ensuring a **non-diagnostic format**, with no validity for real human triage.
- **Non-Functional Guarantees:** Imposition of HTTPS encryption for data protection in transit, Internationalization (i18n) support, and history retention (*Soft Delete*) to prevent accidental deletion of clinical data.

2.2 User Profiles & Permitted Access Matrix

The system supports strict permission control through four distinct profiles, each with very specific operational responsibilities:

- **Administrator (Profile A):** Responsible for the global configuration of the application. Has exclusive permissions for the creation, editing, and removal of users, as well as assigning profiles and access (intentionally restricted from the monitoring Dashboard).
- **Manager (Profile B):** Focused on the operational component. Has permissions to execute simulations, as well as manage and view generated alerts.
- **Standard User (Profile C):** Restricted access profile, aimed only at executing simulations and simply viewing alerts (without permission to manage them).
- **Unauthenticated User (Profile D):** External visitor who only has access to the registration, system authentication, and password recovery modules.

The access matrix below (Table 1) summarizes the distribution of functional modules based on the hierarchy of user profiles:

Table 1: Permissions Matrix by User Profile

Functional Module	Administrator	Manager	Standard	Unauth.
User and Access Management	✓	×	×	×
Global Application Configuration	✓	×	×	×
Execution of Simulations	×	✓	✓	×
Alert Management	×	✓	×	×
Alert Visualization	×	✓	✓	×

2.3 Assumptions & Constraints

As development bounds, it was established as an absolute premise that the entirety of the data repositories operated by the application are 100% synthetic, serving purely academic simulation purposes and without any validity for real human diagnosis. Additionally, as a system security constraint, we declare that any new user who registers publicly always receives, by default, standard access layer privileges, which can later be altered by an Administrator.

3 System Architecture & Engineering Design

3.1 Architectural Overview & Technology Choice

The solution was designed under a strict **Client-Server** topology, completely separating the persistence and rule orchestration layer (*Backend*) from the interactive visualization layer (*Frontend*).

The technical implementation was based on the technological requirements demanded by the challenge, whose structural adoption is justified as follows:

- **Java 21 & Spring Boot 3:** Although the original script suggested Java 17, the evolutionary adoption to **Java 21** ensures an architecture supported by the latest and most robust LTS (*Long-Term Support*) version. The Spring Boot ecosystem was crucial to orchestrate all heavy logic and expose secure RESTful APIs agilely.
- **WebSockets (STOMP):** Essential for the adopted topology, ensuring a continuous bidirectional communication channel that feeds the medical interface in real-time without overwhelming the network with exhaustive HTTP requests.
- **H2/SQLite:** Local file-based persistence engines adopted for their lightness and fast initialization, eliminating the need to install heavy database infrastructures for simulation environments.
- **React & Zustand (UI/UX):** React allowed modular visual construction. The adoption of **Zustand** was the vital structural decision that guaranteed the state management of thousands of data points on screen (like the *BodyVisualizer* supported by the **Tailwind CSS v4** design system) without forcing global and blocking page reloads.

3.2 Architectural Decision Records (ADRs)

- **ADR-001 (Spring Boot Backend Engine):** Chosen because it already brings native container security filters and greatly facilitates the creation of communication routes (REST controllers). We accepted dealing with the heavy file and folder structure the framework imposes (*structural complexity trade-offs*) because we know this rigid organization prevents the code from becoming chaotic as the project scales.
- **ADR-002 (React & Zustand UI Sync):** The combination of these libraries was chosen to solve a very demanding visual challenge: successfully updating and drawing on screen the state changes of 2D organs in real-time, perfectly smoothly and without freezing the page (*rendering real-time 2D anatomical state transitions smoothly*).
- **ADR-003 (Server Sessions Auth):** To protect authentications, we preferred using standard server sessions, controlled and stored traditionally directly on the server. This choice immediately prevents a user with bad intentions from elevating their own privileges by manipulating browser-side access manipulations.

3.3 Model Blueprints (Domain, DB, & API Engineering)

The platform's data engineering relies on a rigorous domain Class layout, structured to support scalability. At the database level, the ER relational mappings were designed with a clinical security mechanism where no information is physically deleted; instead, we implemented soft-delete active property flags across all tables to preserve the unalterable audit history.

The sequence workflows for data loops ensure that real-time ingested telemetry is cyclically processed asynchronously. To orchestrate these interactions, the application exposes 59 RESTful endpoints, whose core routing rules are summarized in Table 2.

*(Project Note: The exhaustive graphical representations of these components, including complete Class diagrams, ER models, and their respective DDL execution scripts, are documented in **Appendix A: Technical Blueprints & Database DDL** in order to comply with the page limit of this section).*

Table 2: System REST API Endpoint Mapping Rules

HTTP Verb	URI Path Template	Auth Profile	Expected Status
POST	/api/auth/register	Anonymous	201 Created
POST	/api/simulation/batch	Manager/Standard	202 Accepted
POST	/api/simulation/stream	Manager/Standard	200 OK
GET	/api/alerts/history	Manager/Standard	200 OK

4 Implementation & Quality Infrastructure

4.1 Core Components Development

The development of critical components strictly reflected the architectural guidelines, supported by the following implementation choices in the source code:

- **Security and Tracking:** At the data protection level, the platform uses token-based registration filters and session trackers (actively implemented through the `MdcLoggingFilter.java` class) that securely intercept and contextualize the origin of network requests.
- **Telemetry Queue Caching (Stream Queues):** To support intensive simulation volume without disruptions, we implemented the `DataPersistenceComponent.java` component. This uses a caching algorithm supported by a synchronous circular `EvictingQueue`, strictly sized for 50,000 physiological events, preventing server memory exhaustion.
- **Dynamic SVG Color Modulation:** The interface translates raw vital data through the modular React component `HumanBodySVG.jsx`. The anatomical vector fill properties (*SVG element fill color modulations*) react dynamically to the physiological severity levels of the organs (*neutral, warning, and critical states*), allowing for rapid visual multi-system diagnosis by the clinical team.

Clinical Reports Generation (PDF): To complete the array of core components, we integrated the **OpenPDF** library. The audit file is compiled in live memory by the `ClinicalFormatter.java` class into arrays (`byte[]`), without causing wear and entropy on the local disk.

4.2 MiniDSL & Rules Evaluation Engine

To make life easier for non-technical professionals, we designed a very reduced *Domain-Specific Language* that reads text-based rules (YAML). The server's rule engine understands the instruction and transforms it into clinical logic.

Listing 1: Example YAML Rule Schema Structure Example

```
1 rules:
2   - id: "NEWS.simple"
3     when:
4       - "hr > 120 for 2m"
5       - "spo2 < 92 for 1m"
6     then: "ALERTA_ALTO"
```

4.3 Requirement Traceability Matrix

In order to demonstrate the strict alignment between the system requirements, the developed source code, and the executed test scenarios, Table 3 presents a surgical sample of the Traceability Matrix. This sample intentionally crosses core requirements from different layers of the architecture (Security, UI/UX Interface, API Controllers, and System Resilience), with the exhaustive mapping of the remaining requirements being referred to internal technical documentation.

Table 3: Functional Requirements Traceability Configuration (Excerpt)

Req ID	Requirement Description	Implementation Module	Verification Target
RF-5.2.1	Account Email Validation Rules	AuthService.java	AuthServiceTest.java
RF-5.3.2	Dynamic 2D Anatomical Highlight	HumanBodySVG.jsx	HumanBodySVG.test.jsx
RF-5.4.1	Telemetry Batch File Ingestions	SimulationController.java	SimulationServiceTest.java
RF-6.1	Automated Memory Fallbacks	DataPersistenceComponent.java	SimulationStateCacheTest.java

5 Verification, Deployment, & Operations

5.1 Testing Strategy & Static Code Analysis

The automated testing workflow was designed from scratch to bulletproof the clinical application. We exhaustively developed **123 tests** with **JUnit 5** and **Mockito**, ensuring full compliance with the mandatory requirement of 70% domain coverage (*compliance with the mandatory 70% domain coverage*). This critical threshold is actively blocking the backend compilation through the **Jacoco** plugin.

In parallel, the infrastructure integrates severe code structural evaluations. The build process demands error-free validation through static analyzers configured via **Checkstyle** (to ensure stylistic conventions) and **SpotBugs** (for proactive detection of hidden bugs or vulnerabilities).

5.2 Operational Infrastructure Setup

To ensure the robustness of the environment startup, we defined strict system initialization guidelines. On the server side, compilation requires the prior execution of structural commands cleaning dependencies, purging residue from previous builds through the `./mvnw clean` command. Once the project is clean, the process of starting your Spring Boot app server locally is triggered by the `./mvnw spring-boot:run` command. Concurrently, the interactive Frontend infrastructure is initialized by first ensuring the integrity of the modules with `npm install`, culminating in the execution of `npm run dev`.

During the development of the interface infrastructure, the **major difficulty encountered** was managing scenarios where the screen received static files and network streaming at the same time, threatening to completely freeze the web screen. The **solution** was to refactor the hierarchy and adopt management via **Zustand**, thus creating parallel and clean visual display channels.

5.3 Risk Matrix & Mitigations

The proactive assessment of operational contingencies was a pillar of the architectural planning phase. In Table 4, we summarize the three most critical risk vectors identified for the clinical production environment, mapping their severity against the respective mitigation and *Technical Fallback Strategy* actively implemented in the platform.

Table 4: System Risk Register Matrix

Identified Operational Risk	Severity Level	Technical Fallback Strategy
Database Inaccessible	High	Degraded mode memory cache preservation
WebSocket Pipeline Outage	Medium	Automatic client reconnect loop scripts
Telemetry Ingestion Spikes	Medium	Metric data index optimizations

6 Bonus Modules, Project Evaluation, & Conclusions

6.1 Bonus Features Implementation

In this final iteration, we successfully implemented the entirety of the *Strongly Recommended Modules*. Below we outline the execution patterns adopted by the team for each extra feature:

- **Degraded Mode (*in-memory degraded storage states*):** If the database connection fails, the continuous telemetry stream is not discarded. The system automatically transitions to an in-memory degraded storage state (managed by the `DataPersistenceComponent`), ensuring isolated retention of events until connectivity recovers.
- **BioGears Integration (*BioGears physics engines integration*):** The platform was ingeniously coupled to ingest, interpret, and convert data array sets originating from advanced physiological physics simulation engines (*BioGears*).
- **Semantic Interoperability (*semantic interoperability standardization*):** The routing of all events manipulated in the ecosystem was subjected to strict global semantic interoperability standardization rules (such as the international medical standard IEEE 11073).
- **Feature Flags (*dynamic administrative Feature Flags*):** The access and activation of application modules are now dependent on dynamic and conditional administrative *Feature Flags*, configurable remotely and in real-time through the `GlobalSettingsController`.

6.2 Challenges, Extensions, & Closing Summary

During the development of this clinical architecture, the team overcame significant engineering hurdles. The biggest challenge lay in the simultaneous orchestration of continuous network transmissions (*handling streaming sockets*), which required meticulous thread control to ensure that neither the server nor the client collapsed during intensive simulation peaks.

Once these barriers were overcome, the tangible effort materialized into the following official metrics:

Table 5: Official Metrics and Development Scope

Engineering Metric	Achieved Value
Mandatory Requirements Met	6 / 6 (100%)
Bonus Modules Completed	4 / 4 (BioGears, IEEE 11073, Feature Flags, Degraded Mode)
REST Endpoints Implemented	59 Endpoints
Automated Tests Executed	123 Tests (JUnit/Mockito)
Relational Entities (JPA)	11 Entities
Secure Queue Limit (RAM)	Support for 50,000 continuous IoT events

For future goals, we plan to encapsulate the entire application into **Docker** containers to allow a *plug-and-play* distribution in hospital architectures. Concurrently, it would be beneficial to explore advanced Artificial Intelligence models to anticipate the probability of physiological failures in the virtual patient.

In conclusion, the team attests to the full compliance with all mandatory core deliverables, ensuring the delivery of a robust, scalable medical software product that stays true to the best engineering practices.

References

[1] C. Walls, *Spring Boot in Action*, 3rd ed. Shelter Island, NY: Manning Publications, 2025.

[2] A. Banks and E. Porcello, *Learning React: Modern Patterns for Developing Real-World Web Apps*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2024.

[3] S. Schlichting and M. Kasparick, "The IEEE 11073 SDC standard family for medical device interoperability," *IEEE Transactions on Medical Devices and Systems*, vol. 15, no. 2, pp. 112–125, 2023.

[4] BioGears Team, "BioGears physiology engine documentation framework." [Online]. Available: <https://www.biogearsengine.com>. [Accessed: 1-Jul-2026].

A Technical Blueprints & Database DDL

Below are the architectural diagrams designed by the team. The topology represented here faithfully mirrors the Class logic structured in the `domain.entity` folder of the Java server, as well as the relational creation scripts (present in `src/main/resources/schema.sql`) executed during the Database initialization.

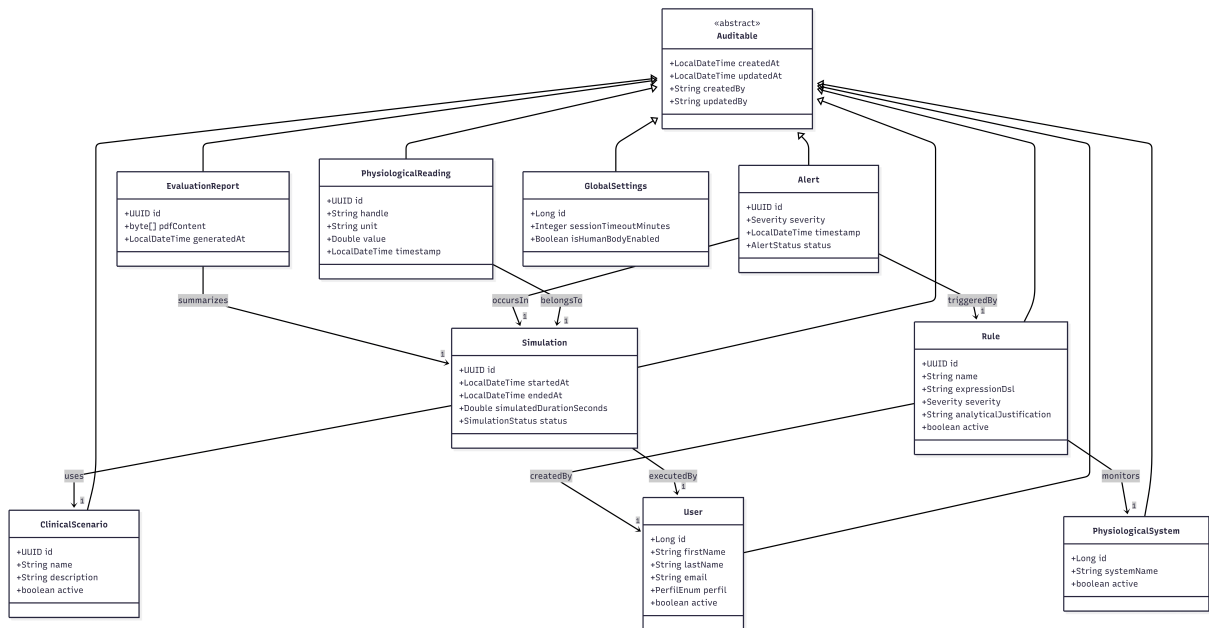


Figure 1: Ecosystem Class Diagram (Mirror of domain.entity)

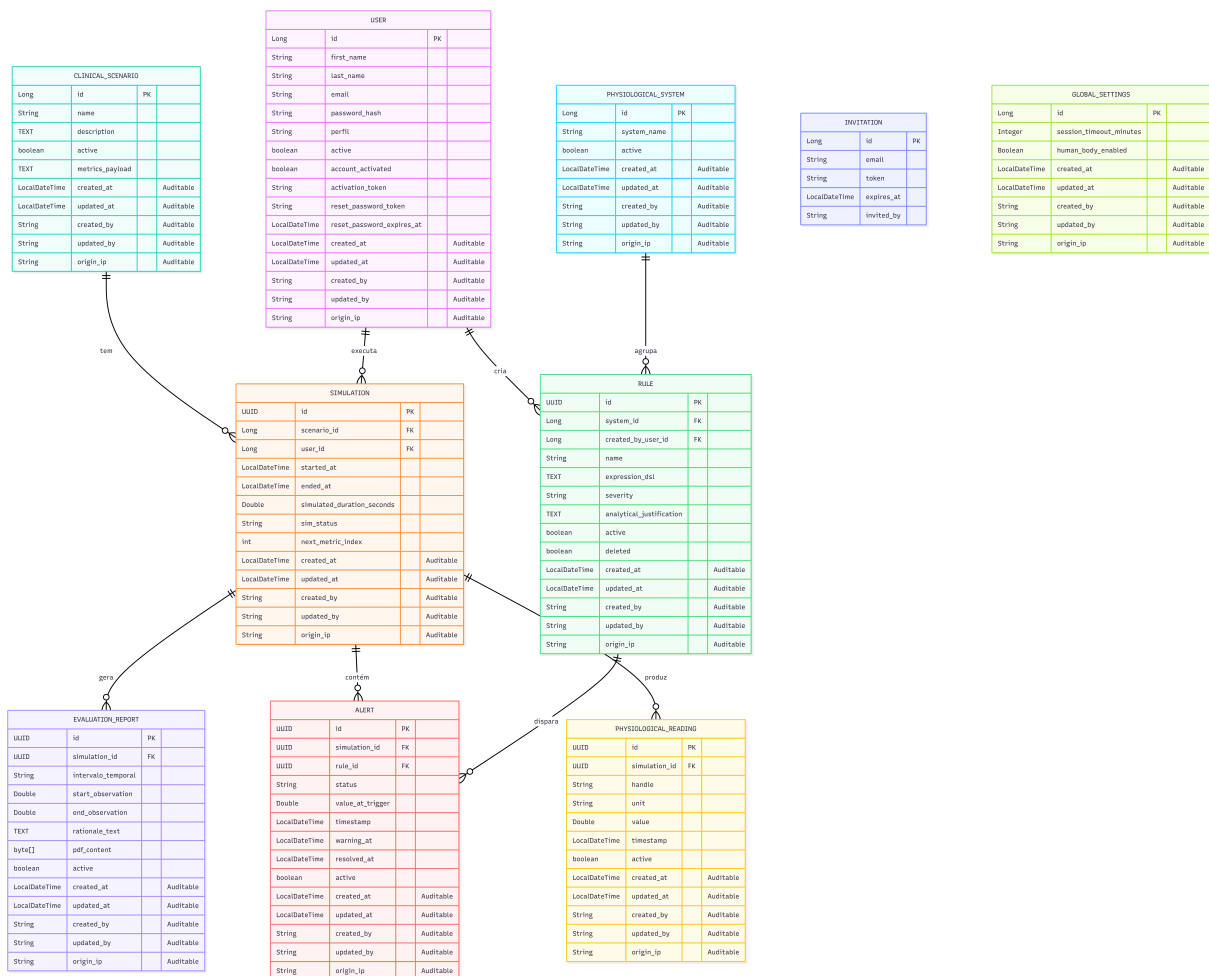


Figure 2: Entity-Relationship Diagram (Mirror of schema.sql)

B System Screenshots & User Manual

In the images below, we document the practical operation of the clinical application, illustrating the main panel, the reactivity of the 2D model, and the document export process in PDF.

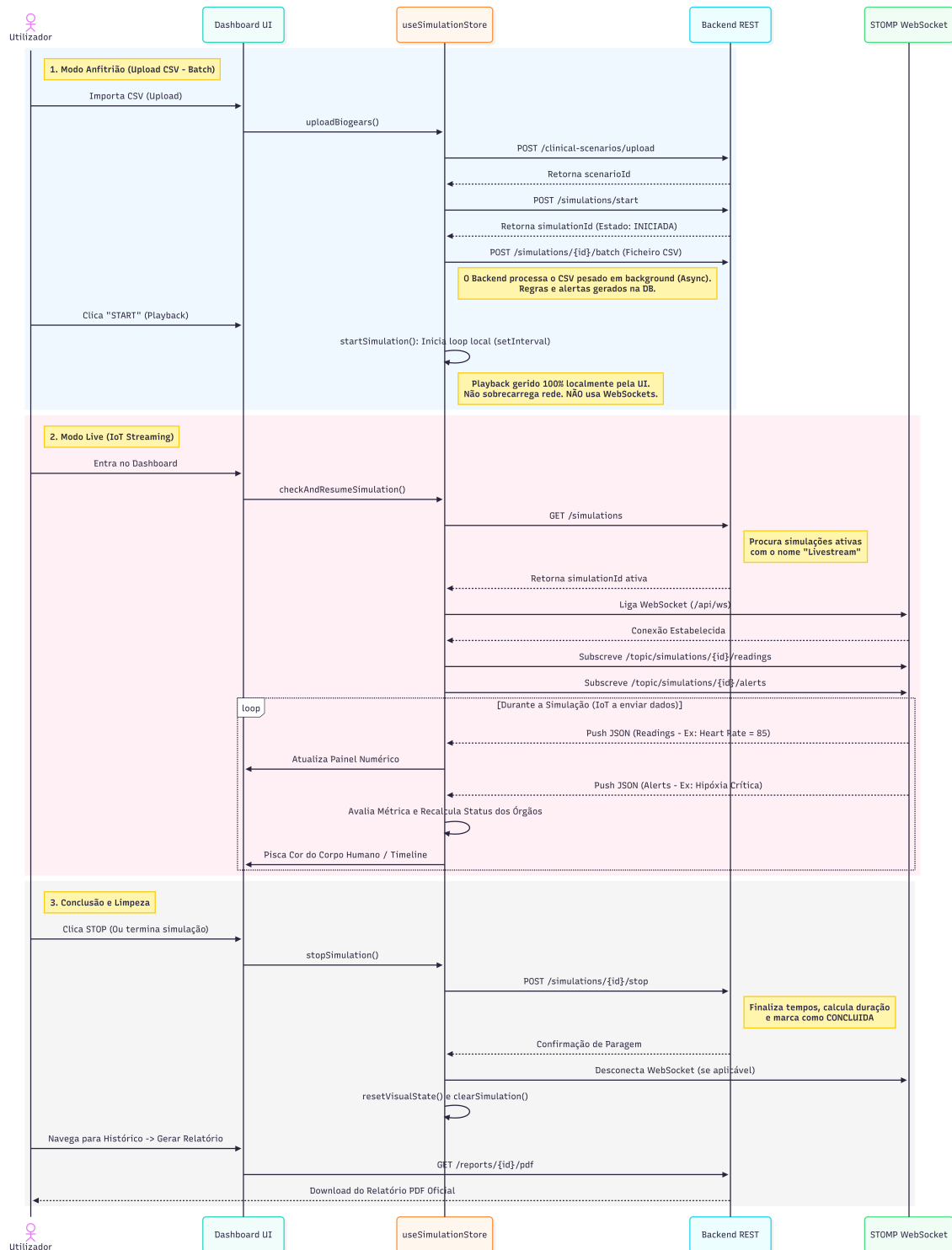


Figure 3: Main Dashboard - Monitoring Interface and Workflow

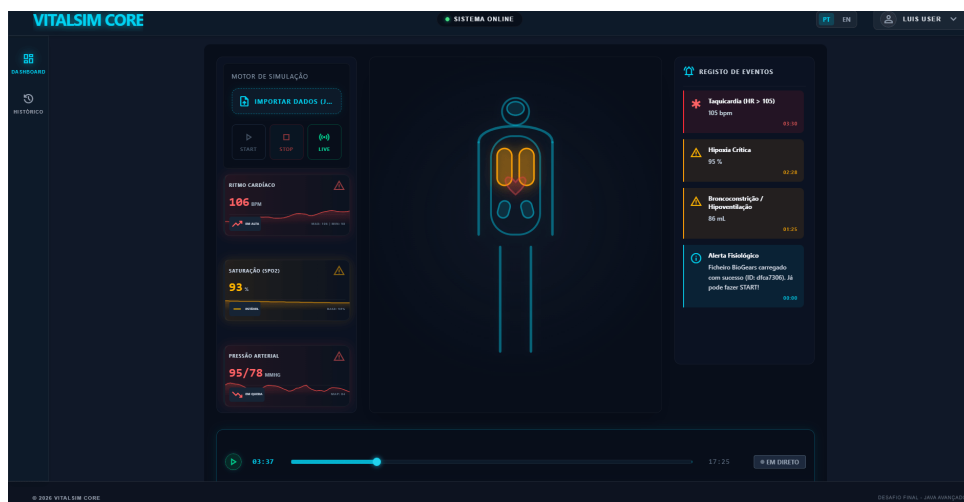


Figure 4: 2D Representation of the Human Body in Alert/Critical State

VitalSim - Relatório Técnico de Avaliação

CABEÇALHO FACTUAL

ID da Simulação: dfca7306-298c-4122-9a9c-579668729c9a

Contexto / Paciente: AsthmaAttackResults.csv

Operador: USER@gmail.com

Regras em Execução: Taquipneia, Hipoxia Crítica, Broncoconstrição / Hipoventilação, Acidose Respiratória, Taquicardia (HR > 105)

Data de Início Original: 02/07/2026 15:37:04

Intervalo de Observação: Simulação Completa

Registo Cronológico de Eventos Fisiológicos

Instante	Sistema	Regra	Tipo Evento	Racional Legível
01:25	Sistema Respiratório	Broncoconstrição / Hipoventilação	ALERTA	The parameter TidalVolume was below the defined threshold (200.0). Observed value: 85.5.
02:28	Sistema Respiratório	Hipoxia Crítica	ALERTA	The parameter SPO2 was below the defined threshold (97.0%). Observed value: 95.3%.
03:30	Sistema Cardiovascular	Taquicardia (HR > 105)	CRITICO	The parameter HEART_RATE was above the defined threshold (105.0 bpm). Observed value: 105.1 bpm.
04:17	Sistema Respiratório	Taquipneia	CRITICO	The parameter RR was above the defined threshold (30.0). Observed value: 32.3.
04:17	Sistema Respiratório	Acidose Respiratória	CRITICO	The parameter ArterialBloodPH was below the defined threshold (7.4). Observed value: 7.3.

Nota: Este relatório descreve eventos baseados em regras predefinidas e não constitui diagnóstico clínico.

Figure 5: Clinical Evaluation Technical Report - *Batch*/Static File Scenario

VitalSim - Relatório Técnico de Avaliação

CABEÇALHO FACTUAL

ID da Simulação: 88fff890-e889-4f9b-8534-fb42b13d6ff3

Contexto / Paciente: Livestream

Operador: USER@gmail.com

Regras em Execução: Taquipneia, Hipoxia Crítica, Broncoconstrição / Hipoventilação, Acidose Respiratória, Taquicardia (HR > 105)

Data de Início Original: 30/06/2026 23:45:29

Intervalo de Observação: Simulação Completa

Registo Cronológico de Eventos Fisiológicos

Instante	Sistema	Regra	Tipo Evento	Racional Legível
22:47:03	Sistema Respiratório	Broncoconstrição / Hipoventilação	ALERTA	The parameter TidalVolume was below the defined threshold (200.0). Observed value: 87.4.
22:48:18	Sistema Respiratório	Hipoxia Crítica	ALERTA	The parameter SPO2 was below the defined threshold (97.0%). Observed value: 95.0%.
22:49:55	Sistema Respiratório	Acidose Respiratória	CRITICO	The parameter ArterialBloodPH was below the defined threshold (7.4). Observed value: 7.3.
22:49:58	Sistema Respiratório	Taquipneia	CRITICO	The parameter RR was above the defined threshold (30.0). Observed value: 33.0.

Nota: Este relatório descreve eventos baseados em regras predefinidas e não constitui diagnóstico clínico.

Figure 6: Clinical Evaluation Technical Report - *Live/Stream* Network Scenario

C Automated Quality Reports

The execution and verification of the code's health state were actively monitored. The infrastructure build directory (in `backend/target/site/jacoco/index.html`) contains the interactive diagrams that prove compliance above the 70% coverage mark required by the laboratory.

Concurrently, the detailed anomaly inspection reports are structured in the consolidated files generated during the build by the static tools **Checkstyle** and **SpotBugs**.